

Chapter 13. Concurrent Performance Techniques

In the history of computing to date, software developers have typically written code in a sequential format. Programming languages and hardware generally only supported the ability to process one instruction at a time. In many situations a so-called “free lunch” was enjoyed, where application performance would improve with the purchase of the latest hardware. The increase in transistors available on a chip led to better and more capable processors.

Many readers will have experienced the situation where moving the software to a bigger or a newer box was the solution to capacity problems, rather than paying the cost of investigating the underlying issues or considering a different programming paradigm.

Moore’s law originally predicted the number of transistors on a chip would approximately double each year. Later the estimate was refined to every 18 months. Moore’s law held fast for around 50 years, but it has started to falter. The momentum we have enjoyed for over 50 years is increasingly difficult to maintain.

The impact of the technology running out of steam can be seen in [Figure 13-1](#), a central pillar of “The Free Lunch Is Over,” a 2005 article written by Herb Sutter that aptly describes the arrival of the modern era of performance analysis.¹

We now live in a world where multicore processors are the norm. Well-written modern applications must take advantage of distributing application processing over multiple cores. Application execution platforms such as the JVM are at a distinct advantage.

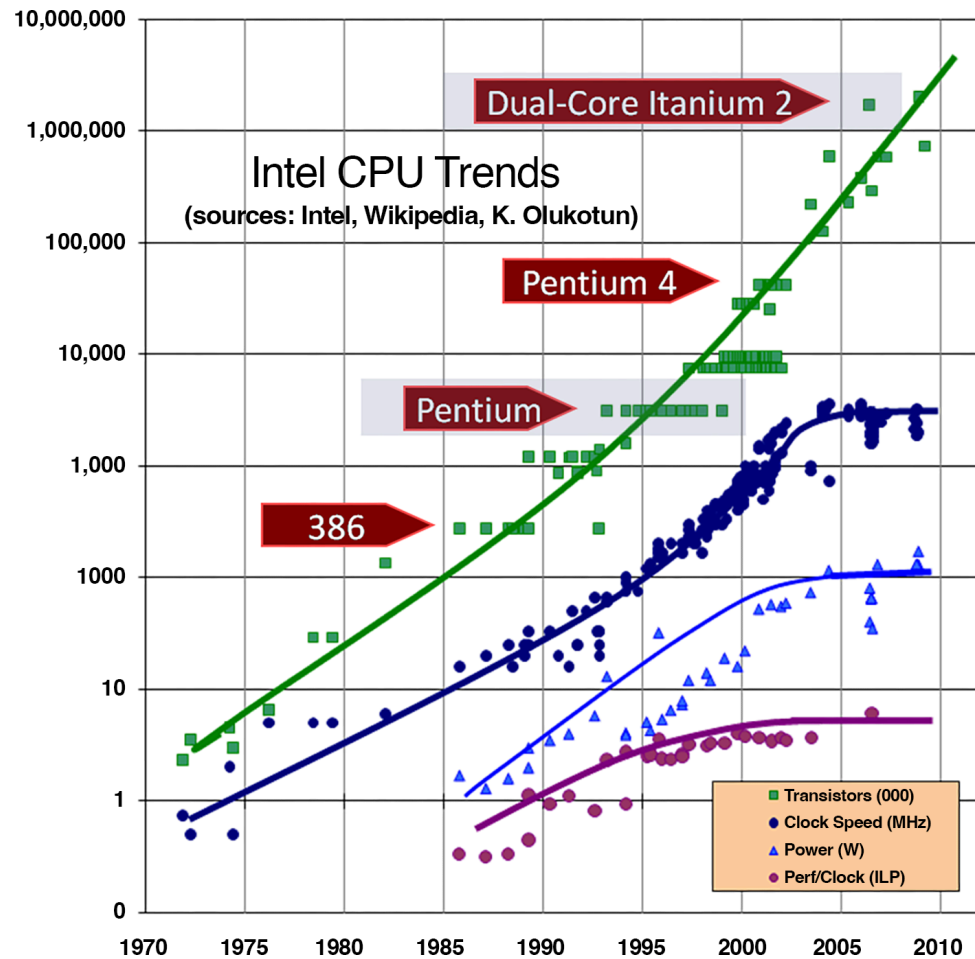


Figure 13-1. The Free Lunch Is Over (Sutter, 2005)

This is because there are always VM threads that can take advantage of multiple processor cores for operations such as JIT compilation and garbage collection. This means even JVM applications that have only a single application thread benefit from multicore hardware.

To make full use of current hardware, the modern Java professional must have at least a basic grounding in concurrency and its implications for application performance. This chapter is a basic overview but is not intended to provide complete coverage of Java concurrency. Instead, a guide such as *Java Concurrency in Practice* by Brian Goetz et al. (Addison-Wesley Professional) should be consulted in addition to this discussion.

Introduction to Parallelism

For almost 50 years, single-core speed increased, and then around 2005 it began to plateau at about 3 GHz clock speed as a result of physical constraints on hardware. Software engineers have been forced to focus more on performance techniques, as only limited improvements to hardware performance can be expected.

In this section, we will discuss some fundamental theoretical underpinnings of parallelism and concurrency.

One of the most important basic concepts is the distinction between *data parallelism* and *task parallelism*.

Data parallelism is about subdividing a single, large task that operates on a large pool of data. This involves distributing chunks of data over different processors, for example, an application that needs to process payroll and can allocate each CPU core a block of employees to process.

Task parallelism, on the other hand, involves distributing execution of different operations over processors, as seen in [Figure 13-2](#). In Java, this will be achieved using threads and `Executor` objects—for example, the pattern whereby each thread services a user in Java REST app.

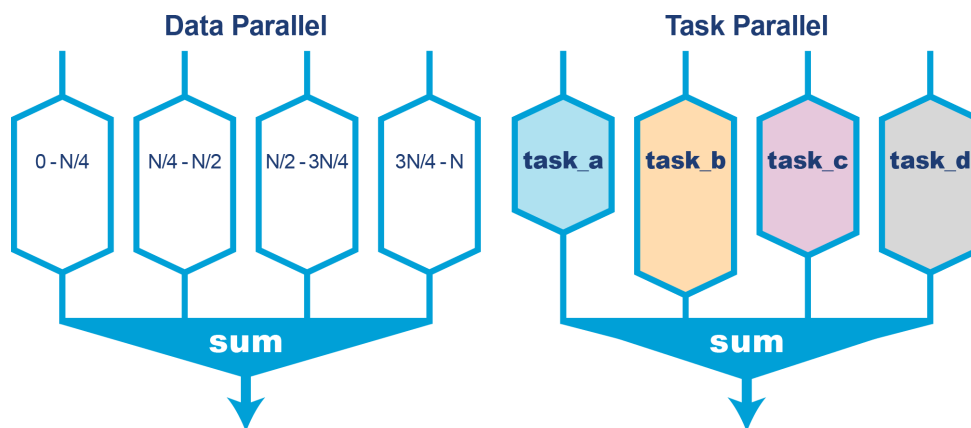


Figure 13-2. Data and task parallel concurrency

It is important to understand when to use each approach, and in this chapter, we will discuss patterns and relevant theory that apply to both cases.

Let's start by meeting a famous law of computation.

Amdahl's Law

In today's multicore world, *Amdahl's law* has emerged as a major consideration for improving the execution speed of a computation task—usually one that is obviously data-parallel.

We introduced Amdahl's law in [Chapter 1](#), but we now need a more formal description. Consider a data-parallel computing task that can be divided into two parts—one part that can be executed in parallel, and one that has to run serially (e.g., for collating results or dispatching units of work for parallel execution).

Let's refer to the serial part as s and the total time needed for the task as T . We can use as many processors as we like for the task, so we denote the number of processors as N . This means we should write T as a function of the number of processors, $T(N)$. The concurrent part of the work

is $T - S$, and if this can be shared equally among N processors, the overall time taken for the task is:

$$T(N) = S + (1 / N) * (T - S)$$

This means that no matter how many processors are used, the total time taken can never be less than the serial time. So, if the serial overhead is, say, 5% of the total, then no matter how many cores are used, the effective speedup will never be more than 20x. This insight and formula make up the underlying theory behind the introductory discussion of Amdahl's law in [Chapter 1](#). The impact can be seen in another way in [Figure 13-3](#).

Only improvements in single-threaded performance, such as faster cores, can reduce the value of S . Unfortunately, trends in modern hardware mean that CPU clock speeds are no longer improving by any meaningful amount. As a consequence of single-core processors no longer getting faster, Amdahl's law is often the practical limit of software scaling.

One corollary of Amdahl's law is that if no communication between parallel tasks or other sequential processing is necessary, then unlimited speedup is theoretically possible. This class of workloads is known as *embarrassingly parallel*, and in this case, concurrent processing is fairly straightforward to achieve.

The usual approach is to subdivide the workload between multiple worker threads without any shared data. Once shared state or data is introduced between threads, the workload increases in complexity and inevitably reintroduces some serial processing and communication overhead.

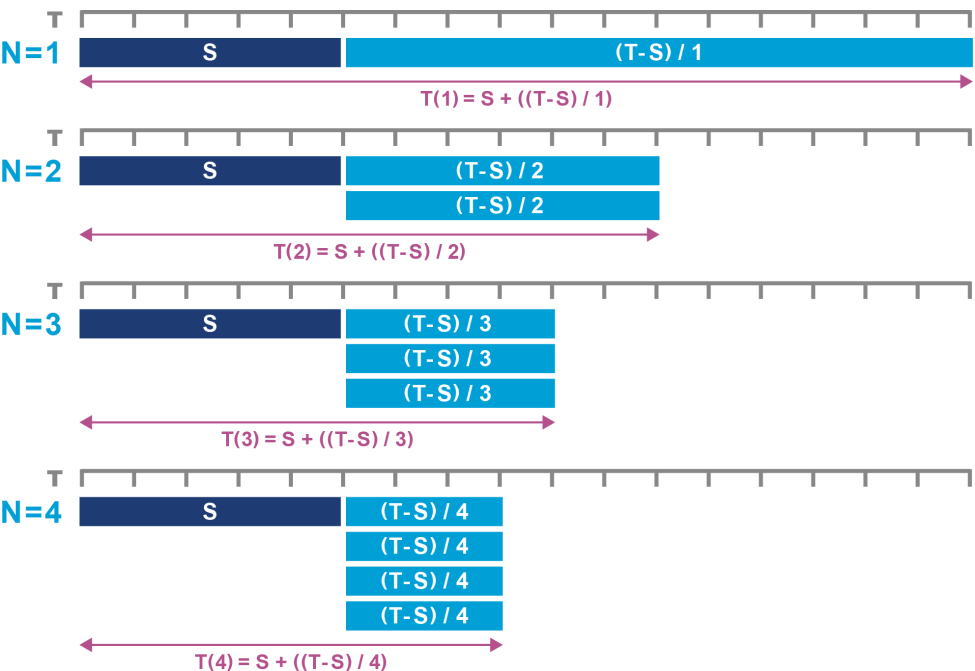


Figure 13-3. Amdahl's law revisited

Writing correct programs is hard; writing correct concurrent programs is harder. There are simply more things that can go wrong in a concurrent program than in a sequential one.

—*Java Concurrency in Practice*, Brian Goetz et al. (Addison-Wesley Professional)

In turn, this means that any workload with shared state requires correct protection and control. For workloads that run on the JVM, the platform provides a set of memory guarantees called the Java Memory Model (JMM). Let's look at some simple examples that explain the problems of Java concurrency before introducing the model in some depth.

Fundamental Java Concurrency

One of the first lessons learned about the counterintuitive nature of concurrency is the realization that incrementing a counter is not a single operation. Let's take a look:

```
public class Counter {
    private int i = 0;
    public int increment() {
        return i = i + 1;
    }
}
```

Analyzing the bytecode for this produces a series of instructions that result in loading, incrementing, and storing the value:

```
public int increment();
Code:
 0: aload_0
 1: aload_0
 2: getfield      #2    // Field i:I
 5: iconst_1
 6: iadd
 7: dup_x1
 8: putfield     #2    // Field i:I
11: ireturn
```

If the counter is not protected by an appropriate lock and is accessed by another thread, it is possible a load could happen before another thread stores the value. This problem results in lost updates.

To see this in more detail, consider two threads, A and B, that are both calling the `increment()` method on the same object. For simplicity's sake, suppose they are running on a machine with a single CPU and that

the bytecode accurately represents low-level execution (so, no reordering, cache effects, or other details of real processors).

NOTE

As the operating system scheduler causes context switching of the threads at non-deterministic times, many different sequences of bytecodes are possible with even just two threads.

Suppose the single CPU executes the bytecodes as shown (note that there is a well-defined order of execution for the instructions, which would not be the case on an actual multiprocessor system):

```
A0: aload_0
A1: aload_0
A2: getfield #2 // Field i:I
A5: iconst_1
A6: iadd
A7: dup_x1
B0: aload_0
B1: aload_0
B2: getfield #2 // Field i:I
B5: iconst_1
B6: iadd
B7: dup_x1
A8: putfield #2 // Field i:I
A11: ireturn
B8: putfield #2 // Field i:I
B11: ireturn
```

Each thread will have a private evaluation stack from its individual entry into the method, so only the operations on fields can interfere with each other (because the object fields are located in the heap, which is shared).

The resulting behavior is that, if the initial state of `i` is `7` before either A or B starts executing, then if the execution order is precisely as just shown, both calls will return `8` and the field state will be updated to `8`, despite the fact that `increment()` was called twice.

WARNING

This issue is caused by nothing other than OS scheduling—no hardware trickery was required to surface this problem, and it would be an issue even on a very old CPU without modern features.

A further misconception is that adding the keyword `volatile` will make the increment operation safe. By forcing the value to always be reread by the cache, `volatile` guarantees that any updates will be seen by another thread. However, it does not prevent the lost update problem just shown, as it is due to the composite nature of the increment operator.

The following example shows two threads sharing a reference to the same counter:

```
public class CounterExample implements Runnable {

    private final Counter counter;

    public CounterExample(Counter counter) {
        this.counter = counter;
    }

    @Override
    public void run() {
        for (int i = 0; i < 100; i++) {
            System.out.println(Thread.currentThread().getName()
                               + " Value: " + counter.increment());
        }
    }
}
```

The counter is unprotected by `synchronized` or an appropriate lock. Each time a program runs, the execution of the two threads can potentially interleave in different ways.

On some occasions the code will run as expected and the counter will increment fine. This is down to the programmer's dumb luck! On other occasions the interleaving may show repeated values in the counter due to lost updates, as seen here:

```
Thread-1 Value: 1
Thread-1 Value: 2
Thread-1 Value: 3
Thread-0 Value: 1
Thread-1 Value: 4
Thread-1 Value: 6
Thread-0 Value: 5
```

In other words, a concurrent program that runs successfully most of the time is not the same thing as a correct concurrent program. Proving it fails is not the same thing as proving it is correct—it is sufficient to find one example of failure to demonstrate it is not correct.

To make matters worse, reproducing bugs in concurrent code can be extremely difficult. Dijkstra’s famous maxim that “testing shows the presence, not the absence of bugs” applies to concurrent code even more strongly than to single-threaded applications.

We could use `synchronized` to control the updating of a simple value such as an `int`.²

The problem with using synchronization is that it requires some careful design and upfront thought. Without this, just adding synchronization to permit concurrency can slow down the program.

This is counter to the whole aim of adding concurrency: to increase throughput. Accordingly, any exercise to parallelize a code base must be supported by performance tests that fully prove the benefit of the additional complexity.

NOTE

Adding synchronization blocks, especially if they are uncontended, is a lot cheaper than it was in older versions of the JVM (but it still should not be done if not necessary).

To do better than just a shotgun approach to synchronization, we need an understanding of the JVM’s low-level memory model and how it applies to practical techniques for concurrent applications.

Understanding the JMM

Java has had a formal model of memory, the JMM, since version 1.0. This model was heavily revised, and some problems were fixed in JSR 133,³ which was delivered as part of Java 5.

In the Java specifications, the JMM appears as a mathematical description of memory. It has a somewhat formidable reputation, and many developers regard it as the most impenetrable part of the Java specification (except, perhaps, for generics).

The JMM seeks to provide answers to questions such as:

- What happens when two cores access the same data?
- When are they guaranteed to see the same value?
- How do memory caches affect these answers?

Anywhere shared state is accessed, the platform will ensure that the promises made in the JMM are honored. These promises fall into two main groups: guarantees related to ordering and those concerned with visibility of updates across threads.

As hardware has moved from single-core to multicore to many-core systems, the nature of the memory model has become increasingly important. Ordering and thread visibility are no longer theoretical issues but are now practical problems that directly impact the code of working programmers.

At a high level, there are two possible approaches that a memory model like the JMM might take:

Strong memory model

All cores always see the same values.

Weak memory model

Cores may see different values, and there are special cache rules that control when this occurs.

From the programming point of view, a strong memory model seems very appealing—not least because it doesn’t require programmers to take any extra care when writing application code.

In [Figure 13-4](#), we can see a (greatly) simplified view of a modern multi-CPU system. We saw this view in [Chapter 5](#) and again in [Chapter 7](#), where it was discussed in the context of NUMA architectures.

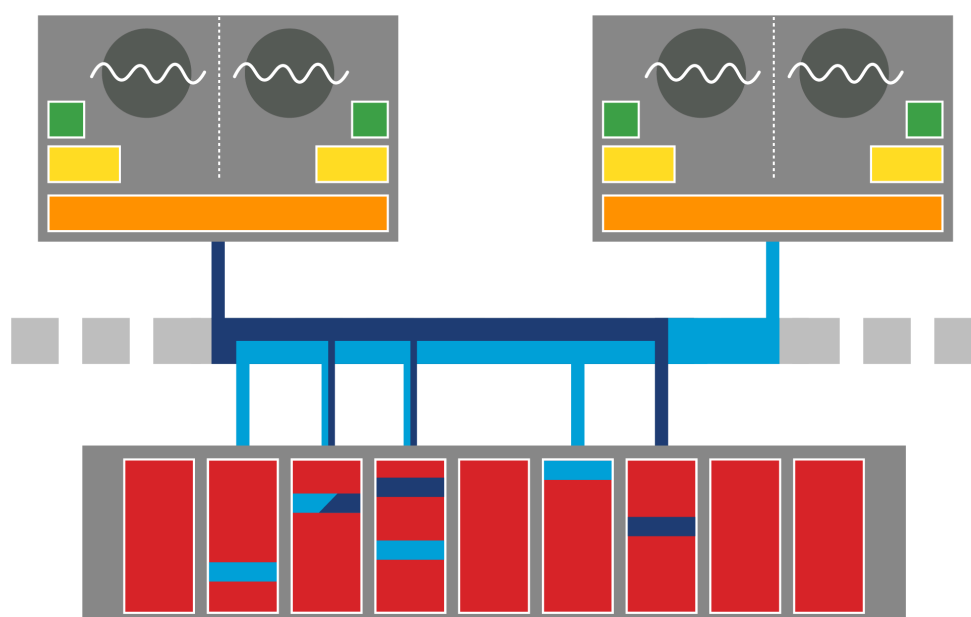


Figure 13-4. Modern multi-CPU system

If a strong memory model were to be implemented on top of this hardware, this would be equivalent to a writeback approach to memory.

Notification of cache invalidation would swamp the memory bus, and effective transfer rates to/from main memory would nosedive. This problem would only get worse as the number of cores increases, which makes this approach fundamentally unsuitable for the many-core world.

It is also worth remembering that Java is designed to be an architecture-independent environment. This means that if the JVM were to specify a strong memory model, it would require additional implementation work to be done in software running on top of hardware that does not support a strong memory model natively. In turn, this would greatly increase the porting work required to implement a JVM on top of weak hardware.

In reality, the JMM has a very weak memory model. This fits better with trends in real CPU architecture, including MESI (described in [“Memory Caches”](#)). It also makes porting easier, as the JMM makes few guarantees.

It is very important to realize that the JMM is a minimum requirement only. Real JVM implementations and CPUs may do more than the JMM requires, as discussed in [“Hardware Memory Models”](#).

This can lead to application developers being lulled into a false sense of security. If an application is developed on a hardware platform with a stronger memory model than the JMM, then undiscovered concurrency bugs can survive—because they do not manifest in practice due to hardware guarantees. When the same application is deployed onto weaker hardware, the concurrency bugs can become a problem as the application is no longer being protected by the hardware.

The guarantees provided by the JMM are based upon a set of basic concepts:

Happens-before

One event definitely happens before another.

Synchronizes-with

The event will cause its view of an object to be synchronized with main memory.

As-if-serial

Instructions appear to execute in order outside of the executing thread.

Release-before-acquire

Locks will be released by one thread before being acquired by another.

One of the most important techniques for handling shared mutable state is locking via synchronization. It is a fundamental part of the Java view of concurrency, and we will need to discuss it in some depth to work adequately with the JMM.

WARNING

For developers who are interested in performance, a passing acquaintance with the `Thread` class and the language-level basic primitives of Java's concurrency mechanism is not enough.

In this view, threads have their own description of an object's state, and any changes made by the thread have to be flushed to main memory and then reread by any other threads that are accessing the same data. This fits well with the write-behind view of hardware as discussed in the context of MESI, but in the JVM, there is a considerable amount of implementation code that wraps the low-level memory access.

From this standpoint, it is immediately clear what the Java keyword `synchronized` refers to: it means that the local view of the thread holding the monitor has been synchronized-with main memory.

Synchronized methods and blocks define touchpoints where threads must perform syncing. They also define blocks of code that must fully complete before other synchronized blocks or methods can start.

The JMM does not have anything to say about unsynchronized access. There are no guarantees about when, if ever, changes made on one thread will become visible to other threads. If such guarantees are required, then the write access must be protected by a synchronized block, triggering a writeback of the cached values to main memory. Similarly, the read access must also be contained within a synchronized section of code to force a reread from memory.

Prior to the arrival of modern Java concurrency, using the Java keyword `synchronized` was the only mechanism of guaranteeing ordering and visibility of data across multiple threads.

The JMM enforces this behavior and offers various guarantees that can be assumed about Java and memory safety. However, the traditional Java `synchronized` lock has several limitations, which have become increasingly severe:

- All `synchronized` operations on the locked object are treated equally. There is not an opportunity to specify priority strategies or differentiate between read and write access.

- Lock acquiring and releasing must be done on a method level or within a `synchronized` block within a method.
- Either the lock is acquired or the thread is blocked; there is no way to attempt to acquire the lock and carry on processing if the lock cannot be obtained.

A very common mistake is to forget that both read and write operations on locked data must be treated equitably. If an application uses `synchronized` only on write operations, this can lead to lost updates.

For example, it might seem as though a read does not need to lock, but it must use `synchronized` to guarantee visibility of updates coming from other threads.

NOTE

Java synchronization between threads is a cooperative mechanism, and it does not work correctly if even one participating thread does not follow the rules.

One resource for newcomers to the JMM is the [JSR-133 Cookbook for Compiler Writers](#). This contains a simplified explanation of JMM concepts without overwhelming the reader with detail.

For example, as part of the treatment of the memory model, a number of abstract barriers are introduced and discussed. These are intended to allow JVM implementors and library authors to think about the rules of Java concurrency in a relatively CPU-independent way.

The rules that the JVM implementations must follow are detailed in the Java specifications. In practice, the actual instructions that implement each abstract barrier may well be different on different CPUs. For example, the Intel CPU model automatically prevents certain reorderings in hardware, so some of the barriers described in the cookbook are actually no-ops.

One final consideration: the performance landscape is a moving target. Neither the evolution of hardware nor the frontiers of concurrency have stood still since the JMM was created. As a result, the JMM's description is an inadequate representation of modern hardware and memory.

In Java 9, the JMM has been extended in an attempt to catch up (at least partially) to the reality of modern systems. One key aspect of this is compatibility with other programming environments, especially C++11, which adapted ideas from the JMM and then extended them. This means that the C++11 model provides definitions of concepts outside the scope of the Java 5 JMM (JSR 133). Java 9 updates the JMM to bring some of those con-

cepts to the Java platform and to allow low-level, hardware-conscious Java code to interoperate consistently with C++11.

To delve deeper into the JMM, see [Aleksey Shipilëv’s blog post “Close Encounters of the Java Memory Model Kind”](#), which is a great source of commentary and very detailed technical information.

Building Concurrency Libraries

Despite being very successful, the JMM is hard to understand and even harder to translate into practical usage. Related to this is the lack of flexibility that intrinsic locking provides.

As a result, since Java 5, there has been an increasing trend toward standardizing high-quality concurrency libraries and tools as part of the Java class library and moving away from the built-in language-level support. In the vast majority of use cases, even those that are performance-sensitive, these libraries are more appropriate than creating new abstractions from scratch.

The libraries in `java.util.concurrent` have been designed to make writing multithreaded applications in Java a lot easier. It is the job of a Java developer to select the level of abstraction that best suits their requirements, and it is a fortunate confluence that selecting the well-abstracted libraries of `java.util.concurrent` will also yield better “thread hot” performance. We use the term “thread hot” to refer to concurrent profiles where threads spend the majority of their time executing and not contending with other threads performing tasks on the same structures.

The core building blocks provided fall into a few general categories:

- Locks and semaphores
- Atomics
- Blocking queues
- Latches
- Executors

In [Figure 13-5](#), we can see a representation of a typical modern concurrent Java application that is built up from concurrency primitives and business logic.

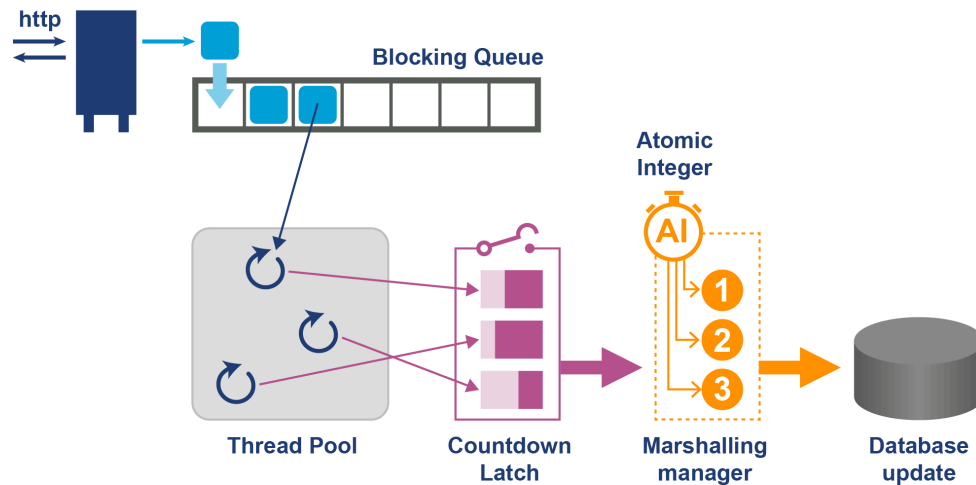


Figure 13-5. Example concurrent application

Some of these building blocks are discussed in the next section, but before we review them, let's look at some of the main implementation techniques used in the libraries. An understanding of how the concurrent libraries are implemented will allow performance-conscious developers to best use them. For developers operating at the extreme edge, knowing how the libraries work will give teams who have outgrown the standard library a starting point for choosing (or developing) ultra-high-performance replacements.

In general, the libraries try to move away from relying upon the operating system and instead work more in user space where possible. This has a number of advantages, not least that the behavior of the library is then hopefully more globally consistent, rather than being at the mercy of small but important variations between Unix-like operating systems.

Method and Var Handles

In [Chapter 6](#), we met `invokedynamic`. This major development in the platform brings much greater flexibility in determining which method is to be executed at a call site. The key point is that an `invokedynamic` call site does not determine which method is to be called until runtime.

Instead, when the call site is reached by the interpreter, a special auxiliary method (known as a *bootstrap method*, or BSM) is called. The BSM returns an object (a *method handle*, provided by the Method Handles API) that represents the actual method that should be called at the call site. This is known as the *call target* and is said to be *laced into* the call site.

NOTE

In the simplest case, the lookup of the call target is done only once—the first time the call site is encountered—but there are more complex cases whereby the call site can be invalidated and the lookup rerun (possibly with a different call target resulting).

At its core, the Method Handles API provides the ability to decide upon, obtain, and call a method at runtime, without any foreknowledge required at compile time. It is similar, in many ways, to the better-known (but much older) Reflection API—however, the API is generally better, less cumbersome, and with several significant design flaws corrected.

NOTE

It's not too much of a stretch to think of method handles as a more modern version of reflection and, as of Java 21, the reflection capability is actually implemented on top of method handles.

A `MethodHandle` object from the package `java.lang.invoke` in `java.base` represents a *directly executable* reference to a method. These method handle objects have a group of several related methods that allow execution of the underlying method. Of these, `invoke()` is the most common, but there are additional helpers and slight variations of the primary invoker method.

Just as for reflective calls, a method handle's underlying method can have any signature. Therefore, the invoker methods present on method handles need to have a very permissive signature to have full flexibility. However, method handles also have a new and novel feature that goes beyond the reflective case.

To understand this new feature, and why it's important, let's first consider some simple code that invokes a method reflectively:

```
Method m = ...
Object receiver = ...
Object o = m.invoke(receiver, new Object(), new Object());
```

This produces the following rather unsurprising piece of bytecode:

```
17: iconst_0
18: new           #2  // class java/lang/Object
21: dup
22: invokespecial #1  // Method java/lang/Object."<init>":()V
25: astore
26: dup
27: iconst_1
28: new           #2  // class java/lang/Object
31: dup
32: invokespecial #1  // Method java/lang/Object."<init>":()V
35: astore
36: invokevirtual #3  // Method java/lang/reflect/Method.invoke
```

```
// : (Ljava/lang/Object;[Ljava/lang/Object;)
// Ljava/lang/Object;
```

The `iconst` and `aastore` opcodes are used to store the zeroth and first elements of the variadic arguments into an array to be passed to `invoke()`. Then, the overall signature of the call in the bytecode is clearly `invoke:(Ljava/lang/Object;[Ljava/lang/Object;)Ljava/lang/Object;`, as the method takes a single object argument (the receiver) followed by a variadic number of parameters that will be passed to the reflective call. It ultimately returns an `Object`, all of which indicates that nothing is known about this method call at compile time—we are punting on every aspect of it until runtime.

As a result, this is a very general call, and it may well fail at runtime if the receiver and `Method` object don't match or if the parameter list is incorrect.

In contrast, let's look at a similar simple example carried out with method handles:

```
MethodType mt = MethodType.methodType(int.class);
MethodHandles.Lookup l = MethodHandles.lookup();
MethodHandle mh = l.findVirtual(String.class, "hashCode", mt);

String receiver = "b";
int ret = (int) mh.invoke(receiver);
System.out.println(ret);
```

There are two parts to the call: first the lookup of the method handle, and then the invocation of it. In real systems, these two parts can be widely separated in time or code location; method handles are immutable stable objects and can easily be cached and held for later use.

The lookup mechanism seems like additional boilerplate, but it is used to correct an issue that has been a problem with reflection since its inception—access control.

When a class is initially loaded, the bytecode is extensively checked. This includes checks to ensure that the class does not maliciously attempt to call any methods that it does not have access to. Any attempt to call inaccessible methods will result in the classloading process failing.

For performance reasons, once the class has been loaded, no further checks are carried out. This opens a window that reflective code could attempt to exploit, and the original design choices made by the reflection

subsystem (way back in Java 1.1) are not wholly satisfactory, for several reasons.

The Method Handles API takes a different approach: the lookup context.

To use this, we create a context object by calling

`MethodHandles.lookup()`. The returned immutable object has state that records which methods and fields were accessible at the point where the context object was *created*.

This means that the context object can either be used immediately, or stored. This flexibility allows for patterns whereby a class can allow selective access to its private methods (by caching a lookup object and filtering access to it). By contrast, reflection only has the blunt instrument of the `setAccessible()` hack, which completely subverts the safety features of Java's access control.

Let's look at the bytecode for the lookup section of the method handles example:

```
0: getstatic      #2 // Field java/lang/Integer.TYPE:Ljava/lang/Class;
3: invokestatic   #3 // Method java/lang/invoke/MethodType.methodType:
                // (Ljava/lang/Class;)Ljava/lang/invoke/MethodType;
6: astore_1
7: invokestatic   #4 // Method java/lang/invoke/MethodHandles.lookup:
                // ()Ljava/lang/invoke/MethodHandles$Lookup;
10: astore_2
11: aload_2
12: ldc           #5 // class java/lang/String
14: ldc           #6 // String hashCode
16: aload_1
17: invokevirtual #7 // Method java/lang/invoke/MethodHandles$Lookup.findV:
                // (Ljava/lang/Class;Ljava/lang/String;Ljava/lang/invo
                // MethodType;)Ljava/lang/invoke/MethodHandle;
20: astore_3
```

This code has generated a context object that can see every method that is accessible at the point where the `lookup()` static call takes place. From this, we can use `findVirtual()` (and related methods) to get a handle on any method visible at that point. If we attempt to access a method that is not visible through the lookup context, then an `IllegalAccessException` will be thrown. Unlike with reflection, there is no way for the programmer to subvert or switch off this access check.

In our example, we are simply looking up the public `hashCode()` method on `String`, which requires no special access. However, we must still use the lookup mechanism, and the platform will still check whether the context object has access to the requested method. Next, let's look at the bytecode generated by invoking the method handle:

```
21: ldc          #8  // String b
23: astore       4
25: aload_3
26: aload        4
28: invokevirtual #9  // Method java/lang/invoke/MethodHandle.invoke
                        // :(Ljava/lang/String;)I
31: istore       5
33: getstatic    #10 // Field java/lang/System.out:Ljava/io/PrintStream;
36: iload        5
38: invokevirtual #11 // Method java/io/PrintStream.println:(I)V
```

This is substantially different from the reflective case because the call to `invoke()` is not simply a one-size-fits-all invocation that accepts any arguments, but instead describes the expected signature of the method that should be called at runtime.

NOTE

The bytecode for the method handle invocation contains better static information about the call site than we would see in the corresponding reflective case.

In our case, the call signature is `invoke:(Ljava/lang/String;)I`, and nothing in the JavaDoc for `MethodHandle` indicates that the class has such a method.

Instead, the `javac` source code compiler has deduced an appropriate type signature for this call and emitted a corresponding call, even though no such method exists on `MethodHandle`. The bytecode emitted by `javac` has also set up the stack such that this call will be dispatched in the usual way (assuming it can be linked) without any boxing of varargs to an array.

Any JVM runtime that loads this bytecode is required to link this method call as is, with the expectation that the method handle will, at runtime, represent a call of the correct signature and that the `invoke()` call will be essentially replaced with a delegated call to the underlying method.

NOTE

This slightly strange feature of the Java language is known as *signature polymorphism* and applies only to method handles.

This is, of course, a very un-Java-like language feature, and the use case is deliberately skewed toward language and framework implementors.

For many developers, one simple way to think of method handles is that they provide a capability similar to reflection but done in a modern way with maximum possible static type safety. Unsurprisingly, they are a very useful toolkit for implementing concurrency and other high-performance libraries.

Historically the only option for implementing low-level concerns has been performed using `Unsafe`. It was the only way to directly access CPU and other hardware features, manage off-heap memory, and generally bypass Java’s well-placed constraints. Despite the intention of `Unsafe` to reside within the use of the JDK, it became ubiquitous in almost all Java frameworks. Methods and var handles are important, as they represent the path forward and underpinnings for concurrency libraries, including atomics and CAS.

Atomics and CAS

Java’s atomic integer class (`java.util.concurrent.AtomicInteger`) has composite operations to add, increment, and decrement, which combine with a `get()` to return the affected result. This means that an operation to increment on two separate threads will return `currentValue + 1` and `currentValue + 2`. The semantics of atomic variables are an extension of `volatile`, but they are more flexible. Thread-based operations are performed without the need to synchronize to guarantee visibility of other interactions.

Note that atomics do not inherit from the base type they wrap, and they do not allow direct replacement. For example, `AtomicInteger` does not extend `Integer`—for one thing, `java.lang.Integer` is (rightly) a final class and, for another, `Integer` represents an immutable value, whereas an `AtomicInteger` is explicitly a thread-safe mutable value.

Atomics, along with some of the other concurrency libraries (notably the locks in `java.util.concurrent`) rely on low-level processor instructions and operating system specifics to implement a technique known as *compare and swap* (CAS).

This technique takes a pair of values, the “expected current value” and the “wanted new value,” and a memory location (a pointer). As an atomic unit, two operations occur:

1. The expected current value is compared with the contents of the memory location.
2. If they match, the current value is swapped with the wanted new value.

CAS is a basic building block for several crucial higher-level concurrency features, so this is a classic example of how the performance and hardware landscape has not stood still since the JMM was produced.

Despite the fact that the CAS feature is implemented in hardware on most modern processors, it does not form part of the JMM or the Java platform specification. Historically, it was, therefore, handled as an implementation-specific extension, and access to CAS hardware was provided via the `sun.misc.Unsafe` class.

However, in recent versions of Java, increasing efforts have been made to remove `Unsafe` and replace it with method and var handles.⁴

NOTE

It is vital for effective use of atomics that developers use the facilities provided and do not roll their own implementations of, say, an atomic integer. Rest assured that the standard library already takes advantage of method handles (and `Unsafe`, where permitted).

Let's look at a quick example that shows how we might approach replacing `Unsafe`:

```
public class AtomicIntegerWithVarHandles extends Number {

    private volatile int value = 0;
    private static final VarHandle V;

    static {
        try {
            MethodHandles.Lookup l = MethodHandles.lookup();
            V = l.findVarHandle(AtomicIntegerWithVarHandles.class, "value",
                               int.class);
        } catch (ReflectiveOperationException e) {
            throw new Error(e);
        }
    }

    public final int getAndSet(int newValue) {
        int v;
        do {
            v = (int)V.getVolatile(this);
        } while (!V.compareAndSet(this, v, newValue));

        return v;
    }
    // ....
}
```

The sample demonstrates the usage of a loop to repeatedly retry a CAS operation. This is to deal with the situation where the comparison fails so the update is not performed. Usually this occurs when another thread just performed an update between the read and the write (as seen by this thread).

This retry loop produces a linear degradation of performance if multiple retries are required to update the variable. When considering performance, it is important to monitor the contention level to ensure throughput levels remain high.

With this caveat, we can see that atomics are lock-free and therefore cannot deadlock.

Locks and Spinlocks

The intrinsic locks that form the basis of `synchronized` in Java operate by invoking the operating system from user code. The OS is used to put a thread into an indefinite wait until signaled. This can be a huge overhead if the contended resource is only in use for a very short time.

Lock-free techniques start from the premise that blocking is bad for throughput and can degrade performance. Instead, it may be much more efficient to have the blocked thread stay active on a CPU, accomplish no useful work, and “burn CPU” retrying the lock until it becomes available.

This technique is known as a *spinlock* and is intended to be more lightweight than a full mutual-exclusion lock. In modern systems, spinlocks are usually implemented with CAS, assuming the hardware supports it. Let’s look at a simple example in low-level x86 assembly:

```
locked:
    dd    0

spin_lock:
    mov    eax, 1
    xchg   eax, [locked]
    test   eax, eax
    jnz    spin_lock
    ret

spin_unlock:
    mov    eax, 0
    xchg   eax, [locked]
    ret
```

The exact implementation of a spinlock varies between CPUs, but the core concept is the same on all systems:

- The “test and set” operation—implemented here by `xchg`—must be atomic.
- If there is contention for the spinlock, processors that are waiting execute a tight loop.

CAS essentially allows the safe updating of a value in one instruction if the expected value is correct. This helps us to form the building blocks for a lock.

Of course, these techniques also come at a cost. Occupying a CPU core is expensive in terms of utilization and power consumption: the machine is going to be non-idle, but this spinning also implies hotter, which means more power will be required to cool the core that’s processing nothing.

Summary of Concurrent Libraries

Concurrency in Java was originally designed for an environment where long-running blocking tasks could be interleaved to allow other threads to execute—for example, I/O and other similar slow operations—but where the underlying hardware often had only one execution core. Nowadays, almost every machine is a multicore system (even mobile phones), so making efficient use of the available CPU resources is very sensible.

However, when the notion of concurrency was built into Java, it was not something the industry had a great deal of experience with. In fact, Java was the first industry-standard environment to build in threading support at language level—with the Thread API. As a result, many of the painful lessons developers have learned about concurrency were first encountered in Java. In Java, the approach has generally been not to deprecate features (especially core features), so the Thread API is still a part of Java and always will be.

This has led to a situation where, in modern application development, threads are quite low level in comparison to the abstraction level at which Java programmers are accustomed to writing code. For example, in Java we do not deal with manual memory management, so why do Java programmers have to deal with low-level threading creation and other lifecycle events?

Fortunately, modern Java offers an environment that enables significant performance to be gained from abstractions built into the language and standard library. This allows developers to have the advantages of con-

current programming with fewer low-level frustrations and less boilerplate.

We've seen an introduction to the low-level implementation techniques used to enable atomic classes and simple locks. Now, let's look at how the standard library uses these capabilities to create fully featured production libraries for general-purpose use.

Locks in `java.util.concurrent`

Java 5 reimaged locks and added a more general interface for a lock in `java.util.concurrent.locks.Lock`. This interface offers more possibilities than the behavior of intrinsic locks:

`lock()`

Traditionally acquires the lock and will block until the lock is available.

`newCondition()`

Creates conditions around the lock, which allows the lock to be used more flexibly. Allows a separation of concerns within the lock (e.g., a read and a write).

`tryLock()`

Tries to acquire the lock (with an optional timeout), allowing for a thread to continue the process in the situation where the lock does not become available.

`unlock()`

Releases the lock. This is the corresponding call following a `lock()`.

In addition to allowing different types of locks to be created, locks can now also span multiple methods, as it is possible to lock in one method and unlock in another. If a thread wants to acquire a lock in a nonblocking manner, it is able to do so using the `tryLock()` method and back out if the lock is not available.

The `ReentrantLock` is the main implementation of `Lock` and basically uses a `compareAndSwap()` with an `int`. This means that the acquisition of the lock is lock-free in the uncontended case. This can dramatically increase the performance of a system where there is less lock contention while also providing the additional flexibility of different locking policies.

TIP

The idea of a thread being able to reacquire the same lock is known as *re-entrant locking*, and this prevents a thread from blocking itself. Most modern application-level locking schemes are re-entrant.

The actual calls to `compareAndSwap()` and the usage of `Unsafe` can be found in the static subclass `Sync`, an extension to `AbstractQueuedSynchronizer`. `AbstractQueuedSynchronizer` also uses the `LockSupport` class, which has methods that allow threads to be parked and resumed. The `LockSupport` class works by issuing permits to threads, and if there isn't a permit available, a thread must wait.

The idea of permits is similar to the concept of issuing permits in semaphores, but here there is only a single permit (a binary semaphore). If a permit is not available, a thread will be parked, and once a valid permit is available, the thread will be unparked. The methods of this class replace the long-deprecated methods of `Thread.suspend()` and `Thread.resume()`.

There are three forms of `park()` that influence the following basic pseudocode:

```
while (!canProceed()) { ... LockSupport.park(this); }
```

They are:

park(Object blocker)

Blocks until another thread calls `unpark()`, the thread is interrupted, or a spurious wakeup occurs.

parkNanos(Object blocker, long nanos)

Behaves the same as `park()` but will also return once the specified nano time elapses.

parkUntil(Object blocker, long deadline)

Is similar to `parkNanos()` but instead uses an absolute point in time (deadline) specified in milliseconds from the Epoch.

Read/Write Locks

Many components in applications will have an imbalance between the number of read operations and write operations. Reads don't change the state, whereas write operations will. Using the traditional

synchronized or ReentrantLock (without conditions) will follow a single lock strategy. In situations like caching, where there may be many readers and a single writer, the data structure may spend a lot of time unnecessarily blocking the readers due to another read.

The `ReentrantReadWriteLock` class exposes a `ReadLock` and a `WriteLock` that can be used within code. The advantage is that multiple threads reading do not cause other reading threads to block. The only operation that will block is a write. Using this locking pattern where the number of readers is high can significantly improve thread throughput and reduce locking. It is also possible to set the lock into “fair mode,” which degrades performance but ensures threads are dealt with in order.

The following implementation for `AgeCache` would be a significant improvement over a version that uses a single lock:

```
public class AgeCache {
    private final ReentrantReadWriteLock rwl = new ReentrantReadWriteLock();
    private final Lock readLock = rwl.readLock();
    private final Lock writeLock = rwl.writeLock();
    private Map<String, Integer> ageCache = new HashMap<>();

    public Integer getAge(String name) {
        readLock.lock();
        try {
            return ageCache.get(name);
        } finally {
            readLock.unlock();
        }
    }

    public void updateAge(String name, int newAge) {
        writeLock.lock();
        try {
            ageCache.put(name, newAge);
        } finally {
            writeLock.unlock();
        }
    }
}
```

However, we could make it even more optimal by considering the underlying data structure. In this example, a concurrent collection would be a more sensible abstraction and yield greater thread hot benefits.

Semaphores

Semaphores offer a unique technique for allowing access to a number of available resources—for instance, threads in a pool or database connection objects. A semaphore works on the premise that “at most, X objects are allowed access” and functions by having a set number of permits to control access:

```
// Semaphore with 2 permits and a fair model
private Semaphore poolPermits = new Semaphore(2, true);
```

`Semaphore::acquire()` reduces the number of available permits by one, and if there are no permits available, will block.

`Semaphore::release()` returns a permit and will release a waiting acquirer if there is one. Because semaphores are often used where resources are potentially blocked or queued, it is most likely that a semaphore will be initialized as fair to avoid thread starvation.

A one-permit semaphore (binary semaphore) is equivalent to a mutex, but with one distinct difference. A mutex can only be released by a thread that the mutex is locked on, whereas a semaphore can be released by a non-owning thread. A scenario where this might be necessary would be forcing the resolution of a deadlock. Semaphores also have the advantage of being able to ask for and release multiple permits. If multiple permits are being used, it is essential to use fair mode; otherwise, there is a chance of thread starvation.

Concurrent Collections

Since Java 5, implementations of the collections interfaces have been specifically designed for concurrent uses. These concurrent collections have been modified and improved over time to give the best possible thread hot performance.

For example, the map implementation (`ConcurrentHashMap`) uses a split into buckets or segments, and we can take advantage of this structure to achieve real gains in performance.

Each segment can have its own locking policy—that is, its own locks. Having both a read and a write lock enables many readers to be reading across the `ConcurrentHashMap`, and, if a write is required, the lock only needs to be held on that single segment. Readers generally do not lock and can overlap safely with `put()` and `remove()`-style operations. Readers will still observe the happens-before ordering for a completed update operation.

It is important to note that iterators (and the spliterators used for parallel streams) are acquired as a sort of snapshot, meaning that they will not throw a `ConcurrentModificationException`. The table will be dynamically expanded when there are too many collisions, which can be a costly operation. It is worthwhile (as with the `HashMap`) to provide an approximate sizing if you know it at the time of writing the code, either as a constant or as a variable.

Java 5 also introduced the `CopyOnWriteArrayList` and `CopyOnWriteArraySet`, which in certain usage patterns can improve multithreaded performance. With these, any mutation operation against the data structure causes a fresh copy of the backing array to be created. Any existing iterators can continue to traverse the old array, and once all references are lost, the old copy of the array is eligible for garbage collection. Again, this snapshot style of iteration ensures that no `ConcurrentModificationException` is raised.

This tradeoff works well in systems where the copy-on-write data structure is accessed for reading many more times than mutating. If you are considering using this approach, make the change with a good set of tests to measure the performance improvement. Given the broad usage of collections, using tools such as JMH and microbenchmarks should be considered and is discussed in [Appendix A](#).

Latches and Barriers

Latches and barriers are useful techniques for controlling the execution of a set of threads. For example, a system may be written where worker threads:

1. Retrieve data from an API and parse it.
2. Write the results to a database.
3. Finally, compute results based on a SQL query.

If the system simply started all the threads running, there would be no guarantee on the ordering of events. The desired effect would be to allow all threads to complete task #1 and then task #2 before starting on task #3. One possibility would be to use a *latch*. Assuming we have five threads running, we could write code like this:

```
public class LatchExample implements Runnable {  
  
    private final CountdownLatch latch;  
  
    public LatchExample(CountdownLatch latch) {  
        this.latch = latch;  
    }  
}
```

```

@Override
public void run() {
    // Call an API
    System.out.println(Thread.currentThread().getName() + " Done API C
    try {
        latch.countDown();
        latch.await();
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    System.out.println(Thread.currentThread().getName()
        + " Continue processing");
}

public static void main(String[] args) throws InterruptedException {
    CountDownLatch apiLatch = new CountDownLatch(5);

    ExecutorService pool = Executors.newFixedThreadPool(5);
    for (int i = 0; i < 5; i++) {
        pool.submit(new LatchExample(apiLatch));
    }
    System.out.println(Thread.currentThread().getName()
        + " about to await on main..");
    apiLatch.await();
    System.out.println(Thread.currentThread().getName()
        + " done awaiting on main..");
    pool.shutdown();
    try {
        pool.awaitTermination(5, TimeUnit.SECONDS);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    System.out.println("API Processing Complete");
}
}

```

In this example, the latch is set to have a count of 5 , with each thread making a call to `countdown()` reducing the number by one. Once the count reaches 0 , the latch will open, and any threads held on the `await()` function will be released to continue their processing.

It is important to realize that this type of latch is single-use only. Once the result is 0 , the latch cannot be reused; there is no reset.

NOTE

Latches are extremely useful in examples such as cache population during start-up and multithreaded testing.

In our example, we could have used two different latches: one for the API results to be finished and another for the database results to complete. Another option is to use a `CyclicBarrier`, which can be reset. However, figuring out which thread should control the reset is quite a difficult challenge and involves another type of synchronization. One common best practice is to use one barrier/latch for each stage in the pipeline.

Executors and Task Abstraction

In practice, most Java programmers should not have to deal with low-level threading concerns (except perhaps for the fire-and-forget use cases of virtual threads). Instead, we should be looking to use some of the `java.util.concurrent` features that support concurrent programming at a suitable level of abstraction. For example, keeping threads busy using some of the `java.util.concurrent` libraries will enable better thread hot performance (i.e., keeping a thread running rather than blocked and in a waiting state).

The level of abstraction that offers few threading concerns can be described as a *concurrent task*—that is, a unit of code or work that we require to run concurrently within the current execution context. Considering units of work as tasks simplifies writing a concurrent program, as the developer does not have to consider the thread lifecycle for the actual threads executing the tasks. This approach also helps with controlled shutdown (i.e., ensuring that threads complete tasks cleanly), as we will see a bit later.

Introducing Asynchronous Execution

One way of fulfilling the task abstraction in Java is by using the `Callable` interface to represent a task that returns a value. The `Callable<V>` interface is a generic interface defining one function, `call()`, that returns a value of type `V` and throws an exception in the case that a result cannot be calculated. On the surface, `Callable` looks very similar to `Runnable`; however, `Runnable` does not return a result and does not throw an exception.

NOTE

If `Runnable` throws an uncaught unchecked exception, it propagates up the stack and, by default, the executing thread stops running.

Dealing with exceptions in the lifetime of a thread is a difficult programming problem. It should also be noted that threads are treated as OS-

style processes, meaning they can be expensive to create on some operating systems. Getting hold of any result from `Runnable` can also add extra complexity, particularly in terms of coordinating the execution return against another thread, for instance.

The `Callable<V>` type provides us with a way to deal with the task abstraction nicely, but how are these tasks actually executed?

An `ExecutorService` is an interface that defines a mechanism for executing tasks on a pool of managed threads. The actual implementation of the `ExecutorService` defines how the threads in the pool should be managed and how many there should be. An `ExecutorService` can take either `Runnable` or `Callable` via the `submit()` method and its overloads.

The helper class `Executors` has a series of `new*` factory methods that construct the service and backing thread pool according to the selected behavior. These factory methods are the usual way to create new executor objects, some of the most common are:

```
newFixedThreadPool(int nThreads)
```

Constructs an `ExecutorService` with a fixed-size thread pool, in which the threads will be reused to run multiple tasks. This avoids having to pay the cost of thread creation multiple times for each task. When all the threads are in use, new tasks are stored in a queue.

```
newCachedThreadPool()
```

Constructs an `ExecutorService` that will create new threads as required and reuse threads where possible. Created threads are kept for 60 seconds, after which they will be removed from the cache. Using this thread pool can give better performance with small asynchronous tasks.

```
newSingleThreadExecutor()
```

Constructs an `ExecutorService` backed by a single thread. Any newly submitted tasks are queued until the thread is available. This type of executor can be useful to control the number of tasks concurrently executed.

```
newScheduledThreadPool(int corePoolSize)
```

Has an additional series of methods that allow a task to be executed at a point in the future that take `Callable` and a delay.

Once a task is submitted, it will be processed asynchronously, and the submitting code can choose to block or poll for the result. The `submit()` call to the `ExecutorService` returns a `Future<V>` that allows a blocking `get()` or a `get()` with a timeout, or a nonblocking call using `isDone()`.

Selecting an ExecutorService

Selecting the right `ExecutorService` allows good control of asynchronous processing and can yield significant performance benefits if you choose the right number of threads in the pool.

It is also possible to write a custom `ExecutorService`, but this is not often necessary. One way in which the library helps is by providing a customization option: the ability to supply a `ThreadFactory`. The `ThreadFactory` allows the author to write a custom thread creator that can set properties on threads such as name, daemon status, and thread priority.

The `ExecutorService` will sometimes need to be tuned empirically in the settings of the entire application. Having a good idea of the hardware that the service will run on and other competing resources is a valuable part of the tuning picture.

One metric typically used is the number of cores versus the number of threads in the pool. Selecting a number of threads to run concurrently that is higher than the number of processors available can be problematic and cause contention. The operating system will be required to schedule the threads to run, and this causes a context switch to occur.

When contention hits a certain threshold, it can negate the performance benefits of moving to a concurrent way of processing. This is why a good performance model and being able to measure improvements (or losses) is imperative. [Chapter 2](#) discusses performance testing techniques and antipatterns to avoid when undertaking this type of testing.

Fork/Join and Parallel Streams

Java offers several different approaches to concurrency that do not require developers to control and manage their own threads. This includes the Fork/Join framework, which provides a new API intended to work efficiently with multiple processors. It is based on a new implementation of `ExecutorService`, called `ForkJoinPool`.

This class provides a pool of managed threads, which has two special features:

- It can be used to efficiently process a subdivided task.
- It implements a *work-stealing* algorithm.

The subdivided task support is introduced by the `ForkJoinTask` class. This is a thread-like entity that is more lightweight than a standard Java thread. The intended use case is that potentially large numbers of tasks and subtasks can be hosted by a small number of actual threads in a `ForkJoinPool` executor.

The key aspect of a `ForkJoinTask` is that it can subdivide itself into “smaller” tasks until the task size is small enough to compute directly. For this reason, the framework is suitable only for certain types of tasks, such as computation of pure functions or other *embarrassingly parallel* tasks. Even then, it may be necessary to rewrite algorithms or code to take full advantage of this part of Fork/Join.

Despite this, the work-stealing algorithm part of the Fork/Join framework can be used independently of the task subdivision.⁵ For example, if one thread has completed all the work allocated to it and another thread has a backlog, it will steal work from the queue of the busy thread. This rebalancing of jobs across multiple threads is a simple but clever idea, yielding considerable benefit.

In [Figure 13-6](#), we can see a representation of work stealing.

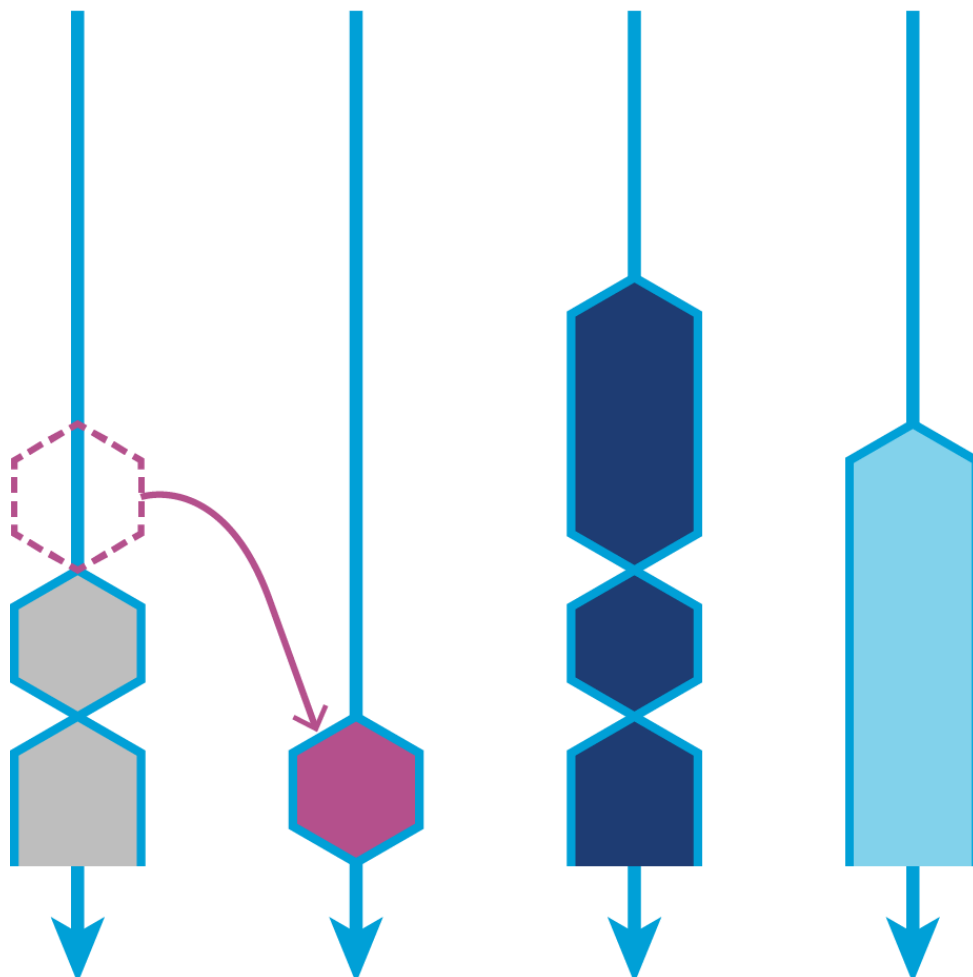


Figure 13-6. The work-stealing algorithm

`ForkJoinPool` has a static method, `commonPool()`, that returns a reference to the system-wide pool. This prevents developers from having to create their own pool and provides the opportunity for sharing. The common pool is lazily initialized so will be created only if required.

The sizing of the pool is defined by

`Runtime.getRuntime().availableProcessors()-1`. However, this method does not always return the expected result.

Writing on the [Java Specialists mailing list](#), Heinz Kabutz found a case where a 16-4-2 machine (16 sockets, each with 4 cores and 2 hyperthreads per core) returned the value 16. This seems very low; the naive intuition gained by testing on our laptops may have led us to expect the value to be $16 \times 4 \times 2 = 128$.

However, if we were to run Java 8 on this machine, it would configure the common Fork/Join pool to have a parallelism of only 15.

The VM doesn't really have an opinion about what a processor is; it just asks the OS for a number. Similarly, the OS usually doesn't care either, it asks the hardware. The hardware responds with a number, usually the number of "hardware threads." The OS believes the hardware. The VM believes the OS.

—Brian Goetz

Thankfully, there is a flag that allows the developer to programmatically set the desired parallelism:

```
-Djava.util.concurrent.ForkJoinPool.common.parallelism=128
```

As discussed in [Chapter 2](#), though, be careful with magic flags. And as we will discuss with selecting the `parallelStream()` option, nothing comes for free!

The work-stealing aspect of Fork/Join is becoming more utilized by library and framework developers, even without task subdivision.

By far the biggest change in Java 8 (arguably the biggest change ever) was the introduction of lambdas and streams. Used together, lambdas and streams provide a sort of “magic switch” to allow Java developers to access some of the benefits of a functional style of programming.

Leaving aside the rather complex question of just how functional Java 8 actually is as a language, we can say that Java now has a new paradigm of

programming. This more functional style involves focusing on data rather than the imperative object-oriented approach that it has always had.

A stream in Java is an immutable sequence of data items that conveys elements from a data source. A stream can be from any source (collection, I/O) of typed data. We operate on streams using manipulating operations, such as `map()`, that accept lambda expressions or function objects to manipulate data. This change from external iteration (traditional `for` loops) to internal iteration (streams) provides us with some nice opportunities to parallelize data and to lazily evaluate complicated expressions.

All collections now provide the `stream()` method from the `Collection` interface. This is a default method that provides an implementation to create a stream from any collection, and behind the scenes a `ReferencePipeline` is created.

A second method, `parallelStream()`, can be used to work on the data items in parallel and recombine the results. Using `parallelStream()` involves separating the work using a `Splitter` and executing the computation on the common Fork/Join pool. This is a convenient technique to work on *embarrassingly parallel* problems, because stream items are intended to be immutable, so they allow us to avoid the problem of mutating state when working in parallel.

The introduction of streams has yielded a more syntactically friendly way of working with Fork/Join than recoding using `RecursiveAction`. Expressing problems in terms of the data is similar to task abstraction in that it helps the developer avoid having to consider low-level threading mechanics and data mutability concerns.

It can be tempting to always use `parallelStream()`, but there is a cost to using this approach. As with any parallel computation, work has to be done to split up the task across multiple threads and then to recombine the results—a direct example of Amdahl's law.

On smaller collections, serial computation can actually be much quicker. You should always use caution and performance-test when using `parallelStream()`. In terms of using parallel streams to gain performance, the benefit needs to be direct and measurable, so don't just blindly convert a sequential stream to parallel.

The arrival of Java 8 also raised the usage level of Fork/Join significantly, as behind the scenes `parallelStream()` uses the common Fork/Join pool.

Actor-Based Techniques

In recent years, several different approaches to representing tasks that are naturally *smaller* than a thread have emerged. We have already met this idea in the `ForkJoinTask` class, and we will meet it again in the section on virtual threads. Another popular approach is the *actor paradigm*.

Actors are small, self-contained processing units that contain their own state, have their own behavior, and include a mailbox system to communicate with other actors. Actors manage the problem of state by not sharing any mutable state and communicating with each other only via immutable messages. The communication between actors is asynchronous, and actors react to the receipt of a message to perform their specified task.

By forming a network in which they each have specific tasks within a parallel system, actors take the view of abstracting away from the underlying concurrency model completely.

Actors can live within the same process, but they are not required to. This opens up a nice advantage that actor systems can be multiprocess and even potentially span multiple machines. Multiple machines and clustering enables actor-based systems to perform effectively when a degree of fault tolerance is required. To ensure that actors work successfully in a collaborative environment, they typically have a fail-fast strategy.

For JVM-based languages, Apache Pekko is a popular framework for developing actor-based systems.⁶ It is written in Scala but also has a Java API, making it usable for Java and other JVM languages as well.

The motivation for an actor-based system is based on several problems that make concurrent programming difficult. The [Pekko documentation](#) highlights three core motivations for considering the use of Pekko over traditional locking schemes:

- Encapsulating mutable state within the domain model can be tricky, especially if a reference to the object's internals is allowed to escape without control.
- Protecting state with locks can cause significant reduction in throughput.
- Locks can lead to deadlock and other types of liveness problems.

Additional problems highlighted include the difficulty of getting shared memory usage correct and the performance problems this can introduce by forcing cache lines to be shared across multiple CPUs.

The final motivation discussed is related to failures in traditional threading models and call stacks. In the low-level threading API, there is no standard way to handle thread failure or recovery. Pekko standardizes this and provides a well-defined recovery scheme for the developer.

Overall, the actor model can be a useful addition to the concurrent developer's toolbox. However, it is not a general-purpose replacement for all other techniques. If the use case fits within the actor style (asynchronous passing of immutable messages, no shared mutable state, and time-bounded execution of every message processor), then it can be an excellent quick win.

If, however, the system design includes request-response synchronous processing, shared mutable state, or unbounded execution, then careful developers may choose to use another abstraction for building their systems.

Let's move on and meet one of the most talked-about new features of Java 21.

Virtual Threads

One of the great strengths of Java is that it is very adaptable—it will take ideas for new features from anywhere. It does so carefully and deliberately, however—the aim is to have the best and most “Javaish” version of a feature, even if it takes more time to arrive.

One of the most important innovations in Java concurrent programming in many years—*virtual threads* (vthreads)—arrived with Java 21 (but the groundwork for them had been laid much earlier). These can be seen as Java's take on *goroutines*, from the Go programming language (or *cooperative processes* in Erlang).

Let's take a first look at them.

Introduction to Virtual Threads

In the very earliest Java versions, the JVM's threads were multiplexed onto OS (aka *platform*) threads in what were referred to as *green threads*.

However, this practice died around the Java 1.2/1.3 era, and modern versions (before Java 21) running on mainstream operating systems basically implement the rule that “one Java thread is exactly one platform thread.”

Calling `Thread.start()` invokes the thread creation system call (e.g., `clone()` on Linux) and actually creates a new OS thread (until then, the

Thread object consists of metadata for a thread that doesn't actually exist yet).

This mechanism—whereby the OS creates, manages, and destroys threads (and processes) has some significant consequences. To see this, recall that the memory space of a process has a standard layout—this goes back to the earliest days of Unix.

Java programs (and the platform threads within them) obey this standard layout and add some specializations within that layout, as can be seen in [Figure 13-7](#).

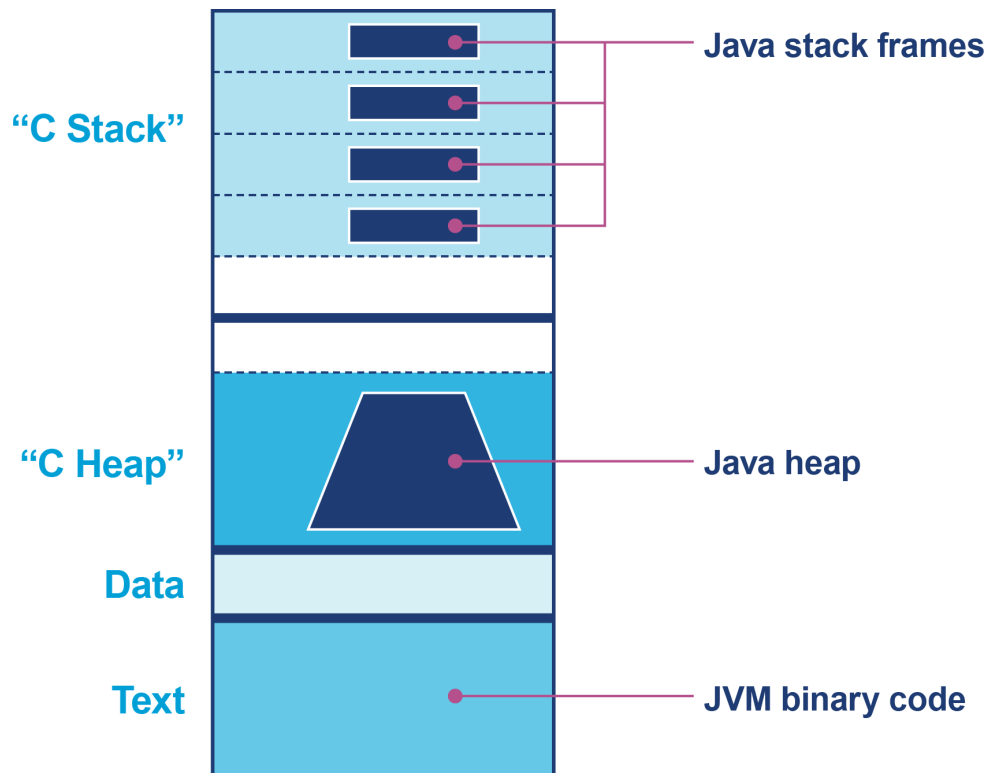


Figure 13-7. Simplified memory layout of a Java process

One aspect of this layout is that OS processes have a *stack segment*. This is a fixed amount of memory that is reserved, per-thread, within the process virtual address space (only one thread's stack segment is shown in the diagram). It is reserved when each thread is created and is not reclaimed until the thread exits.

On Linux x64, the default user space stack size is 1 MB. This means that 1 MB is reserved by the OS each time we launch a new thread.

The math is pretty simple—even for “only” 20,000 threads we need 20 GB of memory. This is an issue—referred to as the *thread bottleneck* problem—especially for thread-per-request and similar architectures.

Virtual threads (which were developed under the codename “Project Loom”) are a response to that problem and attempt to find a solution to it.

That solution can be explained as a question: what if there was a new kind of thread that had these properties:

- Created and managed by the JVM, *not* the OS
- Doesn't have a dedicated platform thread—must share a pool of *carrier threads*
- Replaces static allocation of thread segments with a more flexible model
- Designed for tasks that do (at least some) I/O

Virtual threads are “just” runnable Java objects—they require a platform thread to run upon, but these platform threads are shared as carrier threads. This removes the 1:1 relationship between Java threads and OS threads, and instead establishes a temporary association of a virtual thread to a carrier thread—but this lasts only while the virtual thread is executing.

It also means that when a carrier thread switches between different virtual threads, the context switch may be even cheaper, as there is now no involvement by the operating system. Instead, the switch happens entirely in user space.

Secondly, virtual threads use Java objects within the garbage-collected heap to represent stack frames. This is much more dynamic and removes the static bottleneck caused by stack segment reservation.

To see how we can achieve this, let us consider the thread lifecycle again (as shown in [Figure 7-6](#)). It is certainly possible that threads can run to completion purely in user mode, perhaps doing some calculations, for AI/ML or similar tasks. This means that they would use up their entire CPU timeslice and be swapped out by the OS scheduler. In practice, though, this rarely happens.

Instead, threads often hit a blocking call (e.g., I/O) and switch into kernel space, so the operating system can perform some task on their behalf. Virtual threads use these execution points as a key part of the implementing mechanism.

It's important at this point to be precise about our terminology—Java does not have *bare* syscalls, as it is a fully managed environment. All “system calls” (such as I/O) are actually JDK library calls. Within the implementations of those library calls, the JVM makes syscalls on behalf of the user thread.

Next, remember that Java provides both blocking and nonblocking variants of I/O. As of Java 17, the Java Socket API has been reimplemented in terms of nonblocking I/O (NIO)—previously, it had been based on block-

ing I/O. This did not change the API (only the internals), but it provides an important building block for `vthreads`.

Essentially, every time a `vthread` makes a “blocking” I/O call, it actually performs a nonblocking call instead and yields up its carrier thread. The actual I/O proceeds while the `vthread` is paused, but another `vthread` can now use the carrier thread instead.

There’s nothing particularly special about carrier threads—they’re just a standard Java thread pool (`ExecutorService`), and they show up as standard platform threads to the OS.

NOTE

It is possible to explicitly give up the carrier thread by calling `yield()` from the `vthread`, but this is generally discouraged. As noted in the JavaDoc for `Thread.yield()`, “It is rarely appropriate to use this method” for any type of thread.

It is important to know that code has to specifically create a `vthread`—there is never any “automatic virtualization” of threads. This is important for several reasons, but one of the most important is that the arrival of virtual threads should not change the meaning—or performance characteristics—of any existing code.

In terms of the Java class hierarchy, virtual threads have been added by introducing a new sealed subclass of `Thread`, with a single, final subclass `VirtualThread`, as shown in [Figure 13-8](#).

This accommodates existing code that subclasses `Thread` directly (and subclassing `Thread` always gives a platform thread), while ensuring that all virtual threads are created from a `Runnable`.

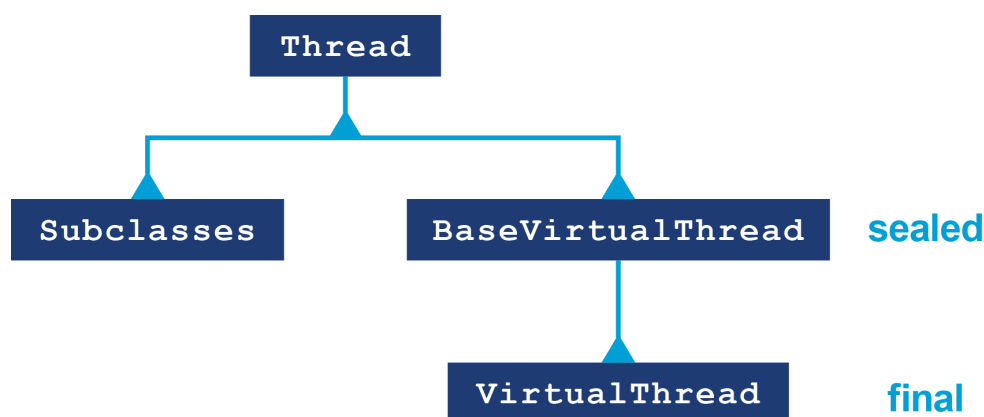


Figure 13-8. New Java thread inheritance hierarchy

To obtain a virtual thread, new static methods have been added to `Thread`, as well as a *Builder* pattern. It can be used like this:

```
Thread.Builder tb = Thread.ofVirtual();
tb.name("MyVirtualThread");
Thread t = tb.unstarted(() -> System.out.println("Hello World!"));
System.out.println(t);
t.start();
```

Note that new methods can be used to get a thread builder object:

`.ofPlatform()` and `.ofVirtual()`. Thread builders can set a name and then be built into either a started or unstarted thread by supplying a `Runnable` task. There are also thread factories available from builders via the `.factory()` method for additional flexibility.

Care has been taken to isolate vthreads—for example, a vthread cannot directly observe its current carrier; `Thread.currentThread()` will return the vthread, and the stack frames from the carrier do not show up in stack trace of vthreads.

NOTE

The JVM language Kotlin has *coroutines*, which might seem superficially similar to vthreads, but they are, in fact, very different. The Kotlin source compiler transforms coroutines into a state machine that is visible in the compiled bytecode, whereas vthreads have support at the Java SDK (and VM) level.

Virtual threads do come with some limitations, including:

- A vthread will yield only if a blocking I/O call is made—there is no preemption.
- JNI calls and the `synchronized` keyword (but not the locks in `java.util.concurrent`) pin a vthread to its carrier and prevent unmounting. When a virtual thread is scheduled, it is mounted or assigned to a platform thread. Unmounting usually occurs at the point where a virtual thread is blocked waiting on I/O or code execution to complete, freeing up the platform thread for other usage. Pinning vthreads in this way can lead to resource issues and unintended blockages.
- vthreads are always daemon threads with normal priority.
- vthreads do not interact well with the Object Pool pattern; vthreads are intended to be short lived, and underlying caching techniques will result in retaining unusable weak references to garbage objects that cannot be reused as intended.

From a tooling perspective, the potentially vast numbers of vthreads can complicate the use of tools such as JMC (and JFR), and new observability patterns will have to be developed to work effectively with them.

NOTE

The story for `ThreadLocal` is more complex for `vthreads`, and it may be better to use the `ScopedValue` approach (which we will meet in [Chapter 15](#)).

Let's conclude this introduction with a few dos and don'ts for `vthreads`:

- Do expect to learn some new intuitions for `vthreads`.
- Don't think of `vthreads` as a free lunch.
- Do learn what types of problems they'll help with—don't just apply them blindly.
- Don't use `vthreads` for compute-bound tasks—they need blocking calls to yield.

In fact, some of the best advice might be to go and talk to other developers—if you have friends who develop in Go, ask them how they use goroutines and what patterns might be transferable.

Virtual Thread Concurrency Patterns

In practice, one of the most immediately obvious benefits of virtual threads is that they should completely remove the need for developers to use the nonblocking form of the NIO APIs directly. Instead, programs can create a dedicated virtual thread that uses the blocking API, and let the runtime sort it out.

The intent is that this is essentially the same thing as using nonblocking I/O in terms of performance, while providing a simpler programming model. Avoiding more complex programming models that display *asynchronous contagion* (such as `async-await` or *colored functions*⁷) was a major design goal for Project Loom.

At the same time, bringing explicit “reactive approaches” into the JDK was a definite non-goal. The end result was the form of virtual threads that we see in Java 21.

Building upon this language feature, we want to examine some relevant patterns, starting with one of the most obvious: just replacing some of a program's threads with `vthreads`.

Recall that carrier threads are threads from a `ForkJoinPool` executor and will yield on (most) blocking operations. This means that for threads that do at least some I/O, there is a potential performance benefit by switching them to virtual. Remember that Java 21 does not do any automatic virtualization—unless you explicitly construct a virtual thread, then you will always get a platform thread.

Of course, as the point of converting some threads to virtual is that we might be able to obtain a performance boost, then we have to test the change—in a real-world complete system—to ensure that we actually realize the expected benefit.

As well as manual creation of virtual threads, there is also a new executor type, which we can get from

`Executors.newVirtualThreadPerTaskExecutor()`. As the name suggests, rather than relying upon a traditional threadpool that is reused for multiple tasks, this executor creates a new virtual thread for each task that is submitted.

To accommodate this new executor type, the `ExecutorService` interface is now `AutoCloseable`—so it can be used in try-with-resources blocks.

This is a great example of a new pattern designed specifically for virtual threads. Executors for platform threads are typically long-lived objects—because they create threads at startup, which is an expensive operation. Therefore, it doesn't make sense to create them as local objects within a method—they're much more likely to be seen as (possibly static) fields.

Virtual threads, on the other hand, are very cheap to create—they're just Java objects. The creation of an executor for virtual threads is similarly cheap, so creating a locally scoped executor doesn't incur the same performance penalty.

This leads to code like this example showing the bare bones of a web server, which uses the block-scoped virtual thread executor:

```
private volatile boolean isShutdown = false;

void handle(Socket socket) {
    // Handle incoming request
}

void serveVT(ServerSocket serverSocket) throws IOException,
    InterruptedException {
    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
        try {
            while (!isShutdown) {
                var socket = serverSocket.accept();
                executor.submit(() -> handle(socket));
            }
        } finally {
            // If there's been an error, or we're interrupted,
            // we stop accepting
            executor.shutdown();
        }
    }
}
```

```

    }
}

public shutdown() {
    isShutdown = true;
}

```

The server socket is passed into the main `serveVT()` method and handles each incoming request by starting a new virtual thread.

Every request is isolated from every other request, so there is no need to share data or context, and the requests will all complete in bounded time (and require network I/O). We can call this type of operation *fire-and-forget*, and this pattern is very suitable for implementing a simple web server using `vthreads`.

It's also important to note that this code uses the *Volatile Shutdown* pattern, which uses the `volatile` field `isShutdown` to force a reread of the flag before accepting any new connection. This is an extremely standard pattern for handling graceful shutdown of Java server applications.

Summary

This chapter only scratches the surface of topics that you should consider before aiming to improve application performance using multithreading. When converting a single-threaded application to a concurrent design:

- Ensure that the performance of straight-line processing can be measured accurately.
- Apply a change and test that the performance is actually improved.
- Ensure that the performance tests are easy to rerun, especially if the size of data processed by the system is likely to change.

Avoid the temptation to:

- Use parallel streams everywhere.
- Create complicated data structures with manual locking.
- Reinvent structures already provided in `java.util.concurrent`.

Aim to:

- Improve thread hot performance using concurrent collections.
- Use access designs that take advantage of the underlying data structures.
- Reduce locking across the application.

- Provide appropriate task/asynchronous abstractions to prevent having to deal with threads manually.

Taking a step back, concurrency is key to the future of high-performance code. However:

- Shared mutable state is hard.
- Locks can be challenging to use correctly and expensive in terms of hardware resources.
- Both synchronized and asynchronous state sharing models are needed.
- The JMM is a low-level, flexible model.
- The thread abstraction is very low level.

The trend in modern concurrency is to move to a higher-level concurrency model and away from threads, which are increasingly looking like the “assembly language of concurrency.” Recent versions of Java have increased the amount of higher-level classes and libraries available to the programmer. On the whole, the industry seems to be moving to a model of concurrency where far more of the responsibility for safe concurrent abstractions is managed by the runtime and libraries.

In the next chapter, we will see how some of the techniques and patterns we have met can be applied to the clustered and distributed case. We will also see what new complications arise when nontrivial network latency and cluster failure states enter the picture, and how we can overcome them.

¹ Herb Sutter, “The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software,” *Dr. Dobbs’s Journal* 30 (2005): 202–210.

² And before Java 5, `synchronized` was the only choice.

³ The Java platform evolves through Java Specification Requests (JSRs) that track enhancements to the platform standards.

⁴ A full treatment of `Unsafe` is outside the scope of this book—especially as it is an API that the OpenJDK team is actively trying to remove. Further details can be found in [this article in Oracle’s Java magazine](#).

⁵ Arguably, it is the more useful of the two capabilities that `Fork/Join` provides.

⁶ `Pekko` was forked from an earlier project (`Akka`) when the original project adopted a non-open source license.

⁷ Named as such in the blogpost [“What Color is Your Function?”](#) although the author incorrectly assumes that Java is planning to support `async-await`.

